AI Code Generation System Design

Assignment Report

Task 1: 데이터 모델링 및 샘플 정의

1.1 도메인 선택

PPT는 시각적 속성(레이아웃, 색상)과 논리적 이벤트(슬라이드 타입 분기, 데이터 조건 처리)가 복잡하게 얽혀 있는 도메인으로, 과제 배경의 "디자인 도구에서 추출된 정형 데이터" 특성과 정확히 일치한다. LangGraph 실무 경험을 바탕으로 파이프라인을 설계했다.

1.2 데이터 정제 및 구조화 전략

자연어 입력은 본질적으로 모호하고 불완전하다. PPTState JSON으로 추상화하기 전 3단계 정제를 거친다.

정제가 필요한 주요 입력 유형

유형	입력 예시	정제 결과
모호한 표현	"깔끔하게 만들어줘"	theme.background="white", layout="minimal"
불완전한 요청	"매출 차트 넣어줘"	chart_type="bar"(기본값), data=null 플래그
충돌하는 요청	"6개 슬라이드, 5개 타입 명시"	명시된 타입 우선, total_slides=5 자동 조정
수정 대상 불명확	"차트 바꿔줘"	1개면 자동 특정, 2개 이상이면 사용자 확인

정제 3단계

- [1단계] 의도 추출: 자연어 → 슬라이드 타입, 테마, 콘텐츠, mode 파악
- [2단계] 결측값 처리: 미지정 필드 → 기본값 적용 또는 null 플래그
- [3단계] 충돌 해소: 명시적 > 암묵적 / 구체적 > 일반적 / 최신 > 이전 우선순위

> **핵심 구조화 원칙: 조건부 로직을 slots 필드로 명시**
>
> LLM이 조건 분기를 스스로 추론하게 두지 않고, slots 필드에 사전 명시하여 "자유 생성"이 아닌 "슬롯 채우기" 방식으로 코드를 생성하게 한다. 이를 통해 출력이 예측 가능해지고 검증 복잡도가 낮아진다.

1.3 Input 샘플: PPTState JSON

자연어는 모호성 제거를 위해 정제 과정을 거쳐 PPTState JSON으로 변환된다. 이 JSON이 LLM 코드 생성 모델에 실제로 입력되는 구조화된 표현(Intermediate Representation)이다. 슬라이드 스펙 전체 구조는 별첨 input_sample.json 참조. 핵심 필드 설명:

필드	역할
type	슬라이드 유형 → 코드 생성 노드 분기 기준
state	슬라이드 흐름 상태 → LangGraph State 관리
slots	조건부 로직 사전 명시 → LLM 슬롯 채우기 지시
mode	create / edit 분기 → 전체 생성 vs 부분 재생성
target_slide_id	수정 대상 특정 → 해당 슬라이드만 재생성

1.4 Output 샘플: React 컴포넌트 코드

LLM이 slots 필드를 참조하여 생성한 결과물. 전체 코드는 별첨 output_sample.jsx 참조.

Task 2: 파이프라인 아키텍처 설계

2.1 전체 시스템 구성

파이프라인은 4개 Phase로 구성된다.

Phase	구성 모듈	역할
Phase 0	Mode Router	자연어 입력 분석 → create / edit 모드 분기
Phase 1	Intent Parser → Schema Abstractor → Design System Loader → JSON Validator	전처리: 자연어 → PPTState JSON 생성 + Reference Component 주입
Phase 2	Slide Dispatcher → [type]_node → Merge / Target Slot → Code Assembly	모델 추론: Reference 슬롯 채우기 코드 생성
Phase 3	AST Validator → Runtime Validator → Semantic Validator	다층 검증: 문법 → 실행 → 의미

2.2 Phase 1: 전처리

- Intent Parser: 자연어에서 슬라이드 타입, 테마, 콘텐츠 키워드, mode 추출
- Schema Abstractor: 추출 결과 → PPTState JSON 생성 (slots 필드 포함)
- Design System Loader: slide type 목록 기반으로 Reference Component 선택적 로드 ★
- JSON Schema Validator: 필수 필드, 타입 검증. 실패 시 Intent Parser 재시도 (max 2회)

> **Design System Loader: 핵심 추가 노드**
>
> 슬라이드 타입별 기준 컴포넌트(Reference Component)를 파이프라인에 주입한다. LLM은 이후 단계에서 "자유 생성"이 아닌 이 Reference를 기준으로 "슬롯 채우기" 방식으로 코드를 생성한다.

2.3 Phase 2: 모델 추론 (LangGraph StateGraph)

- create mode: Slide Dispatcher → 슬라이드 타입별 노드 병렬 실행 → Merge Node → Code Assembly
- edit mode: target_slide_id 해당 노드만 재실행 (Target Slot Node) → Code Assembly

- 각 노드의 LLM 프롬프트: "Reference Component의 {{slot}} 부분을 slots 필드를 참조하여 채워라. 구조는 변경 금지."

2.4 Phase 3: 다층 검증

검증 레이어	도구	검증 내용	실패 시
3-A AST	Babel Parser	JSX 문법, import 완전성, 태그 쌍	Code Assembly 재실행
3-B Runtime	샌드박스 렌더링	런타임 오류, 슬라이드 개수 일치	해당 슬라이드 노드 재실행
3-C Semantic	LLM	Reference 대비 슬롯 누락/일탈 여부	누락 슬롯 명시 후 재생성

2.5 재생성 루프 및 탈출 조건

- revision_count < 3: 자동 재생성 루프 유지
- revision_count >= 3: Human-in-the-loop 진입 → 사용자 확인 요청
- 오류 유형별 분기: AST 실패 → Code Assembly / Runtime 실패 → 슬라이드 노드 / Semantic 실패 → fix_prompt와 함께 재생성

2.6 LangGraph State 구조

```
class PPTState(TypedDict):

  user_request:          str

  mode:                  Literal['create', 'edit']

  target_slide_id:       str | None

  slide_spec:            dict   # PPTState JSON

  reference_components: dict   # Design System Loader 주입

  react_code:            str

  validation_result:     dict

  revision_count:        int
```

Task 3: 핵심 메커니즘 구체화

핵심 모듈: Semantic Validator

3.1 선택 근거

AST 검증(문법)과 Runtime 검증(실행)은 기존 도구로 대체 가능하다. Semantic Validator만이 "의도 충족 여부"를 판단한다.

> **Semantic Validator가 잡는 오류 유형**
>
> slots 필드에 severity 값에 따라 red, yellow, green으로 분기하도록 지시했음에도, LLM이 severity와 무관하게 항상 동일한 accent 색상을 출력하는 경우가 대표적이다. JSX 문법은 올바르고 렌더링도 정상적으로 동작하기 때문에 AST 검증과 Runtime 검증은 통과하지만, 의도한 조건 분기가 구현되지 않았으므로 Semantic 검증에서만 탐지할 수 있다.

3.3 동작 원리 개요

Step	모듈	동작
STEP 1	Slot List Extractor	slide_spec의 slots 필드 순회 → 검증 대상 슬롯 목록 생성 (중요도 분류 포함)
STEP 2	Slide Component Extractor	react_code에서 슬라이드별 컴포넌트만 분리 추출 (토큰 70% 절감)
STEP 3	LLM Slot Verifier	Reference + 생성 코드를 LLM에 전달 → 슬롯 충족 여부 판단
STEP 4	Verdict Generator	pass/fail 판정 + 누락 슬롯 목록 + fix_prompt 생성

3.4 의사 코드 (Pseudo Code)

전체 의사 코드는 별첨 semantic_validator_pseudo.py 참조.

Task 4: 기술 전략 기술서

4.1 설계 선택 이유

왜 PPT 코드 생성 도메인인가

PPT 생성은 시각적 속성(레이아웃, 색상)과 논리적 이벤트(슬라이드 타입 분기, 데이터 조건 처리)가 복잡하게 얽혀 있는 도메인으로, 과제 배경의 "디자인 도구에서 추출된 정형 데이터" 특성과 정확히 일치한다. 실무에서 직접 구현을 시도했다가 검증 레이어의 부재로 중단한 경험이 있으며, 이번 설계는 그 실패 원인을 분석하고 해결한 결과물이다.

왜 자유 생성이 아닌 슬롯 채우기(Slot Filling)인가

LLM에게 "PPT 코드를 생성하라"고 지시하는 자유 생성 방식은 출력 공간이 무한히 넓어 일관성이 없고 검증 기준 정의가 어렵다. Reference Component가 구조를 고정하고 LLM이 내용만 채우는 슬롯 채우기는 출력 공간을 제약하여 일관성을 높이고 검증을 단순화한다. 이는 Few-shot 전략을 코드 생성에 구조적으로 적용한 것이다.

왜 LangGraph인가

이유	설명
State 기반 부분 수정	target_slide_id 해당 노드만 재실행. 전체 재생성 대비 비용 최대 1/6 절감.
조건부 엣지로 검증 루프	AST 실패는 Code Assembly로, Semantic 실패는 슬라이드 노드로 정밀 분기. 단순 Chain으로 구현 불가.
Human-in-the-loop 내장	revision_count 임계값 초과 시 interrupt() 로 사용자 확인 요청. 무한 루프 방지.

왜 Semantic Validator를 핵심 모듈로 선택했는가

AST/Runtime 검증은 기존 도구로 대체 가능하다. Semantic Validator만이 "의도 충족 여부"를 판단한다. 과제가 요구하는 "데이터의 인과관계를 소프트웨어 로직으로 변환하는 신뢰성 확보"가 바로 이 레이어에서 실

현된다.

4.2 예상되는 기술적 제약 사항

제약	원인	대응 전략
Semantic Validator 판단 비일관성	LLM 확률적 특성 (temperature > 0)	temperature=0 고정, Yes/No 바이너리 응답 강제, JSON 포맷 한정
Reference 표현력 한계	미등록 슬라이드 타입 요청 시 처리 불가	자주 요청되는 타입 사전 등록, 미등록 시 유사 Reference fallback
전역 변경 처리 불가	테마 변경 등 모든 슬라이드에 전파 필요	Change Scope Classifier 추가: local/global 구분 후 global은 THEME 객체만 교체
데이터 미입력 필드	정제 단계에서 data=null 플래그 발생	샘플 데이터 대체 + 워터마크 표시, 이후 Edit Mode로 실제 데이터 입력

4.3 정확도를 높이기 위한 전략

단계	전략	효과
생성 전	Design System으로 출력 공간 축소	LLM이 틀릴 수 있는 경우의 수를 구조적으로 감소
생성 중	slots에 type_hint + forbidden 필드 추가	recharts BarChart 사용 강제, 인라인 hex 코드 금지 등 실수 패턴 사전 차단
생성 후	구체적 fix_prompt 전달	누락 슬롯 위치 + 기준 + 구현 방식 명시 → 재생성 1회 해결 확률 극대화
한계 시	Human-in-the-loop	3회 실패 시 사용자 확인 요청 → 시스템 신뢰성 보존

4.4 설계 전체를 관통하는 핵심 철학

> **핵심 원칙**
>
> "LLM이 틀릴 수 있는 공간을 구조적으로 좁히고, 틀렸을 때 정확히 어디서 왜 틀렸는지 알 수 있게 만든다."

생성 전: Design System으로 출력 공간 제약 → 틀릴 확률 낮춤

생성 중: slots + type_hint + forbidden으로 유도 → 올바른 방향 강제

생성 후: AST → Runtime → Semantic 다층 검증 → 틀린 곳 정확히 탐지

실패 시: 구체적 fix_prompt로 재생성 → 재생성 1회 해결

한계 시: Human-in-the-loop → 시스템 신뢰성 보존

개별 LLM의 성능에 의존하지 않고 파이프라인 구조 자체가 신뢰성을 만들어내는 시스템을 목표로 했다.